SRE - Assignment 1

Technical Debt Inventory and Analysis

of the Python-Markdown code base

Group Report

#	Group member	Roll number	Individual files (Appendix A)
1	Taha Yasir	i222429	`extensions/def_list.py`, `extensions/meta.py`
2	Khizar	i222595	`extensions/admonition.py`, `extensions/wikilinks.py`
3	[Member 3 name]	i228780	`postprocessors.py`, `extensions/legacy_attrs.py`

Section: [Section] Instructor: [Instructor] Submitted: [Date]

Contents

1. Project Overview 3
   1.1 Repository and version analysed 3
   1.2 System description and purpose 3
   1.3 Size and module count 3
2. Inventory Analysis 4
   2.1 Project assets 4
   2.2 Classification of assets 4
   2.3 Quality metrics per module 5
   2.4 Dependency mapping 6
3. Tool-Based Metrics 7
   3.1 Cyclomatic complexity 7
   3.2 Cognitive complexity 8
   3.3 Code duplication 8
   3.4 Test coverage 8
   3.5 Bugs, vulnerabilities and code smells 9
   3.6 Maintainability index 9
4. Debt Identification 10
   4.1 Example of code debt: a deeply nested method (TD-01) 10
   4.2 Example of design debt: one fix, two copies (TD-03) 11
   4.3 Example of test debt: tests tied to one library release (TD-04) 11
   4.4 Example of documentation debt (TD-05) 11
   4.5 Example of infrastructure debt (TD-06) 11
5. Remediation Cost and Debt Ratio 12
   5.1 Method 12
   5.2 Tool outputs and cost per item 12
   5.3 Technical Debt Ratio 13
6. Prioritization Strategy 13
   6.1 Technique 1: hotspot analysis 13
   6.2 Technique 2: weighted scoring model 14
   6.3 Prioritised order and justification 15
7. Reflection 15
Appendix A. Individual Component: Manual Calculations 17
   A.1 Member 1: Taha Yasir (i222429) 17
   A.2 Member 2: Khizar (i222595) 19
   A.3 Member 3: [Member 3 name] (i228780) 20
Appendix B. Reproducing the Measurements 21

1. Project Overview

1.1 Repository and version analysed

Item	Value
Project	Python-Markdown (PyPI package `Markdown`)
Repository URL	https://github.com/Python-Markdown/markdown
Commit hash	`0bf535bc406c95dad66050f07c8402af03fe07d5`
Commit date / position	9 October 2026, branch `master`, 7 commits after release tag 3.11.0
Licence	BSD 3-Clause
Language / runtime	Python, `requires-python >= 3.11`; no mandatory third-party runtime dependency
Project age and activity	First commit March 2006; 1,672 commits by 191 contributors; 44 commits in the last 12 months

1.2 System description and purpose

Python-Markdown is the reference Python implementation of John Gruber's Markdown. It converts Markdown text into HTML or XHTML and is the rendering engine behind documentation tools such as MkDocs, so its output is published on a very large number of project web sites. It is used as a library (`markdown.markdown(text)`) and as a command-line tool (`markdown_py`).

The system is a five-stage pipeline coordinated by the `Markdown` class in `core.py`: **preprocessors** clean the raw lines and stash raw HTML, the **block parser** builds an ElementTree from block-level syntax (paragraphs, lists, headers, code), **tree processors** run the **inline patterns** (emphasis, links, images) over the tree, a **serializer** writes the tree as HTML, and **postprocessors** restore the stashed fragments. Every stage keeps its processors in a priority `Registry`, and an **extension API** lets plug-ins register additional processors. Eighteen extensions ship with the package (tables, footnotes, table of contents, fenced code, admonitions and others).

1.3 Size and module count

Only the installable package `markdown/` is treated as business logic. Tests, documentation and build files are inventoried in Section 2 but are not counted in the size below.

Measure (radon raw)	Core (14 modules)	Extensions (19 modules)	Package total
Physical lines of code (LOC)	4,651	3,958	8,609
Source lines (SLOC: no blanks, comments, docstrings)	2,279	2,172	4,451
Logical lines (LLOC: statements)	2,408	2,171	4,579
Functions and methods	207	190	397
Classes	63	48	111

Selection criterion	Requirement	Python-Markdown	Met
Size	5,000 - 12,000 LOC of business logic	8,609 LOC in the package (4,451 SLOC once comments and docstrings are removed)	Yes
Structure	At least 10 classes / modules	33 modules, 111 classes	Yes
Complexity	Non-trivial decision logic	23 functions with cyclomatic complexity above 10; maximum 33	Yes
Language	Java, Python, C#, C++ or React	Python 3	Yes

2. Inventory Analysis

2.1 Project assets

Asset type	Location	Size	Content
Code - core	`markdown/*.py`	14 modules, 4,651 LOC	Pipeline engine: `core`, block parser and processors, inline patterns, tree processors, HTML parser, serializers, `util`, CLI entry point
Code - extensions	`markdown/extensions/`	19 modules, 3,958 LOC	`Extension` base class plus 18 bundled plug-ins
Code - helper scripts	`tools/`	2 files	Formatter used only when the documentation site is built
Tests	`tests/`	37 Python files, 14,667 LOC, 1,123 test cases	`unittest` suites: `test_apis`, `test_extensions`, `test_syntax/` (blocks, inline, extensions), `test_legacy`
Test fixtures	`tests/*/` data folders	300 files (`.txt`, `.text`, `.html`)	Input / expected-output pairs for the legacy file-driven tests
Libraries - runtime	`pyproject.toml`	0 mandatory, 2 optional	Standard library only (20 stdlib modules). Optional: Pygments (code highlighting), PyYAML (CLI config files)
Libraries - development	`pyproject.toml`, `tox.ini`, `uv.lock`	3 test, 8 documentation pins	`coverage`, `pyyaml`, `pytidylib`; docs tool chain pinned to exact versions (`zensical==0.0.62`, `mkdocstrings==1.0.6`, ...)
Configuration / build	repository root	11 files	`pyproject.toml`, `tox.ini`, `makefile`, `MANIFEST.in`, `.coveragerc`, `.codecov.yml`, `mkdocs.yml`, `.pyspelling.yml`, `.spell-dict`, `checklinks.sh`, `uv.lock`
CI / process	`.github/`	6 workflows, 1 PR template	Test matrix (tox), changelog enforcement and validation, version check, documentation deploy
Documentation	`docs/`, root `*.md`	68 Markdown pages (about 5,800 lines) + 4 root files	User guide, extension reference, extension-API guide, 665-line changelog; README, INSTALL, LICENSE, CODE_OF_CONDUCT
UI	`markdown/__main__.py`, `.zensical/`	1 CLI module (154 LOC), 16 theme files	No graphical UI. User-facing surfaces are the `markdown_py` command line and the documentation web-site theme

2.2 Classification of assets

Each asset was classified from three sources of evidence: whether the default pipeline imports it (dependency graph), whether the code marks it with `@deprecated` or "legacy", and how many other modules reuse it (fan-in).

Asset	Class	Evidence
`core.py`, `__init__.py`, `__meta__.py`, `__main__.py`	Active	Entry points and orchestrator; `core` imports all eight pipeline modules
`blockparser.py`, `blockprocessors.py`, `inlinepatterns.py`, `treeprocessors.py`, `preprocessors.py`, `htmlparser.py`	Active	Always on the default conversion path; 31 of the 71 package commits of the last three years touched these six files
`postprocessors.py`	Active (part deprecated)	Two active processors; `UnescapePostprocessor` is marked `@deprecated`
`util.py`	Reusable	Generic `Registry`, `deprecated` decorator, constants; imported by 16 modules
`extensions/__init__.py`	Reusable	`Extension` base class - the plug-in contract; imported by 20 modules and by every third-party extension
`serializers.py`	Reusable	Generic ElementTree-to-HTML writer adapted from ElementTree 1.3; no dependency on the rest of the package
`test_tools.py`	Reusable	`TestCase` helpers published for authors of third-party extensions

Asset	Class	Evidence
16 extensions: `abbr`, `admonition`, `attr_list`, `codehilite`, `def_list`, `extra`, `fenced_code`, `footnotes`, `md_in_html`, `meta`, `nl2br`, `sane_lists`, `smarty`, `tables`, `toc`, `wikilinks`	Active	Registered as entry points in `pyproject.toml`; documented and tested
`extensions/legacy_attrs.py`, `extensions/legacy_em.py`	Deprecated (legacy)	Kept only so that documents written for versions before 3.0 still render
`AbbrPreprocessor`, `AbbrInlineProcessor` (abbr), `get_name`, `stashedHTML2text` (toc), `get_attrs` (attr_list), `Markdown.docType` (core)	Deprecated	`@deprecated` decorator, "soft-deprecated" docstring, or a TODO saying the attribute is unused
`tests/test_syntax/`, `test_apis.py`, `test_extensions.py`	Active	Current style: one test class per feature
`tests/test_legacy.py` and its 300 fixture files	Deprecated (legacy)	Older file-comparison tests, kept until they are ported to `test_syntax`
Documentation, CI workflows, build configuration	Active	Updated in step with releases; changelog is enforced by a CI workflow

2.3 Quality metrics per module

LOC and SLOC from radon; CC = cyclomatic complexity (radon); Cog = cognitive complexity (complexipy); MI = maintainability index (radon, 0-100, higher is better); coverage from coverage.py; Dup = lines inside duplicated blocks (jscpd); Churn = commits that touched the file in the last three years.

Module	LOC	SLOC	Fn	CC sum	CC max	Cog max	MI	Stmt %	Branch %	Dup	Churn
`__init__.py`	48	5	0	0	0	0	100.0	100.0	100.0	0	0
`__main__.py`	154	103	2	8	5	6	70.6	100.0	100.0	0	2
`__meta__.py`	51	14	1	6	6	3	75.9	100.0	100.0	0	13
`blockparser.py`	160	34	7	12	5	10	50.2	100.0	90.0	0	1
`blockprocessors.py`	642	346	34	113	11	18	34.6	97.7	94.4	50	5
`core.py`	531	197	12	41	10	14	48.9	100.0	94.7	32	5
`extensions/__init__.py`	145	34	7	14	3	2	53.3	100.0	90.0	0	2
`extensions/abbr.py`	199	115	15	37	10	18	59.5	93.3	91.2	16	11
`extensions/admonition.py`	192	98	7	29	13	19	62.4	100.0	100.0	23	4
`extensions/attr_list.py`	213	116	12	51	32	102	58.5	100.0	93.5	9	6
`extensions/codehilite.py`	355	178	9	40	14	28	58.9	99.1	92.9	7	4
`extensions/def_list.py`	128	63	5	21	17	19	69.9	96.6	100.0	12	3
`extensions/extra.py`	66	18	3	3	1	0	100.0	100.0	100.0	0	0
`extensions/fenced_code.py`	201	133	7	35	24	65	60.0	99.0	95.2	7	4
`extensions/footnotes.py`	494	312	33	80	7	19	40.3	97.3	84.8	22	6
`extensions/legacy_attrs.py`	78	27	5	11	7	9	79.3	100.0	100.0	12	2
`extensions/legacy_em.py`	59	20	2	2	1	0	89.2	100.0	100.0	34	1
`extensions/md_in_html.py`	458	291	22	123	25	94	32.9	98.0	92.0	38	9
`extensions/meta.py`	93	47	4	13	10	17	78.4	100.0	90.0	7	4
`extensions/nl2br.py`	49	10	2	2	1	0	100.0	100.0	100.0	7	2
`extensions/sane_lists.py`	77	26	4	4	1	0	83.2	100.0	100.0	7	3
`extensions/smarty.py`	295	151	10	17	5	4	65.8	100.0	78.6	8	6
`extensions/tables.py`	255	155	10	53	14	26	51.6	100.0	98.6	28	4
`extensions/toc.py`	496	323	26	98	14	25	37.0	94.5	91.7	11	5
`extensions/wikilinks.py`	105	55	7	13	5	7	78.7	100.0	100.0	11	3
`htmlparser.py`	460	290	28	101	14	19	36.5	99.0	90.0	9	15
`inlinepatterns.py`	1052	524	48	142	33	58	25.9	99.5	94.9	36	11
`postprocessors.py`	131	58	9	16	5	10	76.0	96.0	100.0	0	3
`preprocessors.py`	91	30	4	4	1	0	82.4	100.0	100.0	0	1
`serializers.py`	214	124	8	47	24	75	59.2	99.1	97.0	24	2
`test_tools.py`	224	93	9	21	7	28	68.4	91.5	94.4	0	0

Module	LOC	SLOC	Fn	CC sum	CC max	Cog max	MI	Stmt %	Branch %	Dup	Churn
`treeprocessors.py`	483	282	19	101	17	73	40.1	97.8	93.7	0	6
`util.py`	410	179	26	51	9	9	52.2	96.0	97.4	0	5
`Package total / worst`	**8,609**	**4,451**	**397**	**1,309**	**33**	**102**	**25.9**	**98.1**	**93.4**	**205**	**71**

2.4 Dependency mapping

The import graph was extracted from the abstract syntax tree of every module (`analysis/deps.py`). Imports made only for type checking (`if TYPE_CHECKING:`) were kept apart because they do not exist at run time.

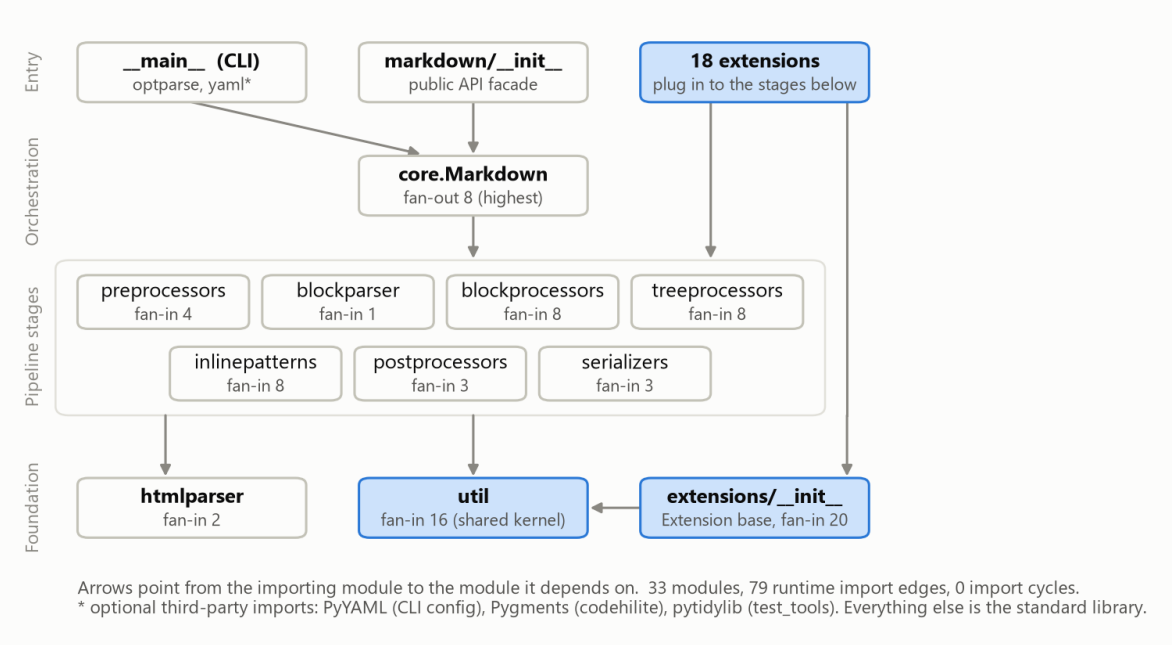


Figure 1. Layered view of the internal dependencies

Module	Fan-in (Ca)	Fan-out (Ce)	Instability I = Ce / (Ca + Ce)	Depends on
`extensions`	20	1	0.05	util
`util`	16	0	0.00	(nothing internal)
`blockprocessors`	8	2	0.20	blockparser, util
`inlinepatterns`	8	1	0.11	util
`treeprocessors`	8	2	0.20	inlinepatterns, util
`preprocessors`	4	2	0.33	htmlparser, util
`core`	1	8	0.89	blockprocessors, extensions, inlinepatterns, postprocessors, preprocessors, serializers, treeprocessors, util
`extensions.fenced_code`	0	6	1.00	extensions, extensions.attr_list, extensions.codehilite, preprocessors, serializers, util
`extensions.footnotes`	0	6	1.00	blockprocessors, extensions, inlinepatterns, postprocessors, treeprocessors, util
`extensions.md_in_html`	0	6	1.00	blockprocessors, extensions, htmlparser, postprocessors, preprocessors, util

- **No cycles.** 33 modules, 79 run-time import edges and no import cycle. A further 20 edges exist only under `TYPE_CHECKING`; 14 of them point back to the top-level package and would be cycles if they were real imports.
- **Stable foundation, unstable edge.** `util` (I = 0.00) and the `Extension` base (I = 0.05) are depended on by almost everything and depend on almost nothing, while every extension has I = 1.00. Dependencies therefore point towards stability, which is the healthy direction.

- **Concentration risk.** `util` is a shared kernel with fan-in 16: a change to `Registry` or to the placeholder constants can affect half of the code base.
- **Cross-extension coupling.** `fenced_code` imports two other extensions (`attr_list`, `codehilite`), and `wikilinks` reads the attribute `Markdown.Meta` that only the `meta` extension creates. The second link is invisible to the import graph.
- **External dependencies.** Run-time code imports only the standard library. Three third-party packages are imported optionally, each by a single module: Pygments (`codehilite`), PyYAML (`__main__`), pytidylib (`test_tools`).

3. Tool-Based Metrics

The assignment allows SonarQube or any other tool. SonarQube Community needs a Docker or Java server, and we could not keep one running on the analysis laptop, so we used open-source analysers that each measure one of the required metrics. Section 5 applies the same SQALE formula that SonarQube uses for remediation cost and debt ratio. All commands and raw outputs are in the `analysis/` folder (Appendix B).

Metric	Tool (version)	Result
Size	radon raw (6.0.1)	8,609 LOC, 4,451 SLOC, 33 modules
Cyclomatic complexity	radon cc (6.0.1)	total 1,309; mean 3.30 per function; maximum 33; 23 functions above 10
Cognitive complexity	complexipy (8.0.1)	total 1,743; mean 4.56; maximum 102; 26 functions above 15
Code duplication	jscpd (5.4.0)	2.38 % of lines (205 lines in 26 clone pairs); 4.87 % of tokens
Test coverage - statement	coverage.py (7.16.2)	98.07 % (3,514 of 3,583 statements)
Test coverage - branch	coverage.py (7.16.2)	93.44 % (1,211 of 1,296 branches)
Bugs	pylint (4.1.2) error / fatal classes	0 errors, 0 fatal; 20 reliability warnings (Section 3.5)
Vulnerabilities	bandit (1.9.4)	0 high, 0 medium, 25 low severity
Code smells	pylint, flake8 (7.4.1), vulture (2.16)	761 pylint messages (554 convention, 93 refactor, 114 warning); pylint score 8.03 / 10
Maintainability index	radon mi (6.0.1)	mean 63.0; SLOC-weighted 47.4; lowest 25.9; every module in rank A

3.1 Cyclomatic complexity

radon reports 397 functions and methods with a total cyclomatic complexity of 1,309. The mean is 3.30 and the median is 2, so the typical function is simple: 336 functions (85 %) are in rank A. The debt sits in the tail: 23 functions exceed the usual threshold of 10 and 9 exceed 15.

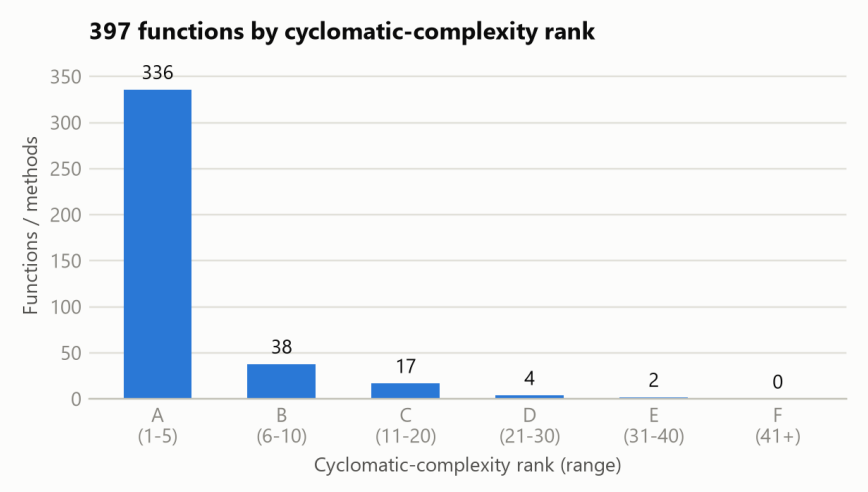


Figure 2. Distribution of cyclomatic complexity (radon ranks)

3.2 Cognitive complexity

Cognitive complexity adds a penalty for nesting, so it separates code that merely has many branches from code that is hard to follow. The total is 1,743 (mean 4.56). 26 functions exceed SonarQube's default limit of 15 and 14 exceed 25. The table lists every function above 25 with both measures.

Function	Module	Lines	Cyclomatic	Cognitive
`AttrListTreeprocessor.run`	`extensions/attr_list.py`	103-161	32	102
`HTMLExtractorExtra.handle_endtag`	`extensions/md_in_html.py`	160-246	25	94
`_serialize_html`	`serializers.py`	119-192	24	75
`InlineProcessor.__processPlaceholders`	`treeprocessors.py`	172-251	12	73
`FencedBlockPreprocessor.run`	`extensions/fenced_code.py`	91-171	24	65
`LinkInlineProcessor.getLink`	`inlinepatterns.py`	772-886	33	58
`HTMLExtractorExtra.handle_starttag`	`extensions/md_in_html.py`	110-158	23	41
`MarkdownInHtmlProcessor.parse_element_content`	`extensions/md_in_html.py`	328-399	14	37
`InlineProcessor.__applyPattern`	`treeprocessors.py`	253-335	17	36
`InlineProcessor.run`	`treeprocessors.py`	348-419	16	35
`AsteriskProcessor.parse_sub_patterns`	`inlinepatterns.py`	645-704	11	34
`LegacyTestMeta.__new__`	`test_tools.py`	147-193	7	28
`CodeHilite.hilite`	`extensions/codehilite.py`	135-191	14	28
`TableProcessor.test`	`extensions/tables.py`	51-85	14	26

`AttrListTreeprocessor.run` shows why both measures are needed: its cyclomatic complexity of 32 is close to that of `LinkInlineProcessor.getLink` (33), but its cognitive complexity is almost double (102 against 58) because its branches are nested up to six levels deep.

3.3 Code duplication

jscpd (minimum clone size 5 lines and 50 tokens) found 26 clone pairs covering 205 of 8,609 lines, a duplication rate of **2.38 %** (4.87 % of tokens). This is below SonarQube's default gate of 3 %, but several clones are copies of logic that must stay identical:

Clone	Location A	Location B	Why it matters
HTML escaping of &, <, >	`serializers.py` 70-78 and 85-92	`extensions/toc.py` 120-127; `util.py` 158-168	Three copies of output-escaping logic; a fix in one copy can be missed in the others
Workaround for CPython bug gh-77057	`htmlparser.py` 372-380	`extensions/md_in_html.py` 292-302	Temporary fix duplicated in the two most-changed files
Split a block around a match and re-queue the rest	`blockprocessors.py` 594-606	`extensions/abbr.py` 167-179; `extensions/footnotes.py` 286-300	Same parsing idiom re-implemented per processor
Attribute escaping	`serializers.py` 83-94	`serializers.py` 102-113	`_escape_attrib` and `_escape_attrib_html` differ in one line
Processor registration docstring table	`extensions/tables.py` 233-243	`extensions/wikilinks.py` 45-54 and most extensions	Boilerplate; low risk

3.4 Test coverage

The project's own suite (1,123 `unittest` cases) was run under coverage.py with branch measurement. Statement coverage is **98.07 %** (69 of 3,583 statements missed) and branch coverage is **93.44 %** (85 of 1,296 branch outcomes missed, 75 branches only partly taken). The project's `.coveragerc` leaves `test_tools.py` out of its own coverage report; we measured it because it ships in the package. Without it the figures are 98.2 % and 93.4 %, so the choice does not change the result.

These figures needed two runs that were then combined, exactly as the project's CI does. The tests for code highlighting compare exact HTML produced by Pygments and skip themselves unless Pygments 2.7.1 (a 2020 release) is installed and announced through an environment variable. In an ordinary environment 111 tests (9.9 %) are skipped and `extensions/codehilite.py` drops to 11 % coverage (overall 94 %). We treat this as test debt in Section 4.

Module (lowest branch coverage)	Statement %	Branch %	Missed statements	Missed branches
`extensions/smarty.py`	100.0	78.6	0	3
`extensions/footnotes.py`	97.3	84.8	7	14
`blockparser.py`	100.0	90.0	0	1
`extensions/__init__.py`	100.0	90.0	0	1
`extensions/meta.py`	100.0	90.0	0	1
`htmlparser.py`	99.0	90.0	2	7
`extensions/abbr.py`	93.3	91.2	7	3
`extensions/toc.py`	94.5	91.7	14	9

3.5 Bugs, vulnerabilities and code smells

Category	Tool and rule	Count	Assessment
Bugs (confirmed)	pylint error / fatal	0	No defect that a static tool can prove
Bugs (potential)	pylint `super-init-not-called`	8	Subclasses of `Treeprocessor`, `Extension`, `Pattern` skip the base initialiser
	pylint `arguments-differ`, `arguments-renamed`	6	Overrides change the base signature, e.g. `InlineProcessor.handleMatch` takes 3 parameters, the base method 2
	pylint `inconsistent-return-statements`	5	A function returns a value on some paths and `None` on others
	pylint `deprecated-method`	1	`logger.warn()` at `blockprocessors.py:490`
Vulnerabilities	bandit B405 (import of `xml.etree`)	20	False positives: the library builds element trees and never parses XML input (no `fromstring` or `parse` call exists)
	bandit B101 (`assert` in production code)	5	Low: checks vanish under `python -O`
Code smells	pylint convention	554	Mostly `line-too-long` (196), `invalid-name` (152), `consider-using-f-string` (82), missing docstrings (62)
	pylint refactor	93	`no-else-return` (24), `too-many-branches` (12), `too-many-nested-blocks` (9), `duplicate-code` (6)
	pylint warning (remaining)	98	`attribute-defined-outside-init` (36), `redefined-builtin` (29), `unused-argument` (12)
	flake8 (project configuration)	4	Whitespace in one f-string. However, 17 of the 33 modules contain a `flake8: noqa` comment that switches flake8 off for the whole file (Section 4.4)
	vulture (dead code, 80 % confidence)	1	One unused variable in `test_tools.py`

3.6 Maintainability index

radon computes the maintainability index from Halstead volume, cyclomatic complexity, SLOC and comment ratio, scaled to 0-100. All 33 modules fall in rank A (above 19), with a mean of 63.0. The mean hides a wide spread: weighted by size the index is 47.4, and seven modules, which hold 47 % of the lines, score below 41.

Module (lowest MI)	MI	LOC	Functions	CC sum
inlinepatterns.py	25.9	1052	48	142
extensions/md_in_html.py	32.9	458	22	123
blockprocessors.py	34.6	642	34	113
htmlparser.py	36.5	460	28	101
extensions/toc.py	37.0	496	26	98
treeprocessors.py	40.1	483	19	101
extensions/footnotes.py	40.3	494	33	80

4. Debt Identification

The weaknesses measured in Sections 2 and 3 map to ten debt items covering six debt types. Each item cites the evidence that supports it.

ID	Debt type	Weakness	Evidence
TD-01	Code debt	Over-complex functions	26 functions with cognitive complexity above 15, 14 above 25; worst `AttrListTreeprocessor.run` (CC 32, cognitive 102)
TD-02	Design debt	Oversized module with mixed responsibilities	`inlinepatterns.py`: 1,052 lines, 44 functions, 24 classes, lowest MI (25.9), second most-changed file
TD-03	Design debt	Duplicated logic	26 clone pairs, 205 lines; escaping helper exists three times; bug workaround copied into two files
TD-04	Test debt	Environment-pinned tests and branch gaps	111 tests skip unless Pygments is exactly 2.7.1; 85 uncovered branch outcomes; `smarty` 78.6 % and `footnotes` 84.8 % branch coverage; 88 `pragma: no cover` exclusions
TD-05	Documentation debt	Undocumented or misleading code	62 missing docstrings (17 in `htmlparser.py`); 29 % of public definitions have none; stale TODOs; typos in docstrings
TD-06	Infrastructure / dependency debt	Stale platform workarounds and pins	Workaround marked "until we drop support for Python < 3.13" in two files; TODO for bpo-41989; CLI built on `optparse`; docs tool pinned to a 0.0.x release
TD-07	Design debt (obsolescence)	Deprecated and dead code still shipped	10 items: 5 `@deprecated` APIs, 2 legacy extensions, unused `docType` attribute, unused private member, deprecated `logger.warn`
TD-08	Code debt	Inconsistent naming in the public API	152 `invalid-name` findings: camelCase (`extendMarkdown`, `handleMatch`) beside snake_case (`parse_content`)
TD-09	Code debt	Accumulated lint findings and suppressed checks	281 other pylint findings; 17 modules exempted from flake8 by a file-wide noqa comment, hiding 62 over-length lines
TD-10	Security debt	Assertions as run-time checks	5 `assert` statements in production paths (`htmlparser.py`, `serializers.py`)

4.1 Example of code debt: a deeply nested method (TD-01)

`AttrListTreeprocessor.run` in `extensions/attr_list.py` (lines 103-161) decides where an attribute list such as `{: #id .class }` sits in an element. One 59-line method handles six placements, and the same four-line action appears three times at nesting depth five:

```
119    if pos is None and elem[-1].tail:
120        # use tail of last child. no `ul` or `ol`.
121        m = RE.search(elem[-1].tail)
122        if m:
123            if not self.assign_attrs(elem, m.group(1), strict=True):
124                elem[-1].tail = elem[-1].tail[:m.start()]
125    elif pos is not None and pos > 0 and elem[pos-1].tail:
126        # use tail of last child before `ul` or `ol`
127        m = RE.search(elem[pos-1].tail)
128        if m:
129            if not self.assign_attrs(elem, m.group(1), strict=True):
130                elem[pos-1].tail = elem[pos-1].tail[:m.start()]
131    elif elem.text:  ...  # the same block a third time, on elem.text
```

The result is a cyclomatic complexity of 32 and a cognitive complexity of 102, almost seven times SonarQube's limit. Four of its branch outcomes are still untested (93.5 % branch coverage). Extracting "find the text that may carry attributes" from "apply and strip them" would remove the repetition and most of the nesting.

4.2 Example of design debt: one fix, two copies (TD-03)

The workaround below exists in both `htmlparser.py` (line 372) and `extensions/md_in_html.py` (line 292), because the extension subclasses the parser and overrides the same method instead of calling a shared helper:

```
if self.rawdata[i:i+3] == '<![' and not self.rawdata[i:i+9] == '<![CDATA[':
    # We have encountered the bug in #1534 (Python bug `gh-77057`).
    # Provide an override until we drop support for Python < 3.13.
    result = self.parse_bogus_comment(i)
    if result == -1:
        self.handle_data(self.rawdata[i:i + 1])
        return i + 1
    return result
```

These are two of the five most-changed files of the last three years, and 11 of the 15 commits to `htmlparser.py` were bug fixes. Duplicated code in exactly that area means every parser fix has to be made twice. The same pattern appears in the escaping helpers: `serializers._escape_cdata`, `toc.escape_cdata` and `util.code_escape` are three copies of the same replacement logic.

4.3 Example of test debt: tests tied to one library release (TD-04)

```
# tests/test_syntax/extensions/test_code_hilite.py
required_pygments_version = os.environ.get('PYGMENTS_VERSION', '')

class TestCodeHiliteClass(TestCase):
    def setUp(self):
        if has_pygments and pygments.__version__ != required_pygments_version:
            self.skipTest(f'Pygments=={required_pygments_version} is required')
```

The expected HTML in these tests was captured from Pygments 2.7.1, so the tests cannot run against the version users actually install. A developer who runs the suite locally sees it pass while 111 tests are silently skipped and `codehilite.py` is only 11 % covered. A regression caused by a newer Pygments would not be detected. Asserting on structure (tag and class names) instead of on exact markup would remove the pin.

4.4 Example of documentation debt (TD-05)

`htmlparser.py` is the most frequently changed file, yet 18 of its 28 functions have no docstring, including `handle_starttag`, `handle_endtag` and `parse_starttag` where the recent fixes were made. Elsewhere the documentation exists but misleads:

- `core.py:119`: `self.docType = ""  # TODO: Maybe delete this. It does not appear to be used anymore.` The attribute is part of the public `Markdown` object and nobody has resolved the question.
- `postprocessors.py:22, 49, 96`: "after is has been serialized", "after the ElementTree it converted back", "Check is block of HTML is block-level".
- `extensions/admonition.py:159`: "Sibling is a list item, but we need to wrap it's content should be wrapped in <p>".
- Seventeen modules contain a comment such as `# flake8: noqa: E501 116-119`. It reads as an exemption for four long lines, but flake8 has no line-range syntax. We tested it: with the comment present flake8 7.4.1 reports nothing for the file, not even an unused import added as a probe; with the comment removed it reports all of them. Half of the package is therefore not checked by the project's own linter.

4.5 Example of infrastructure debt (TD-06)

The package declares `requires-python >= 3.11`, but `htmlparser.py:169` still carries "Temp fix for bugs.python.org/issue41989 - TODO: remove this when the bug is fixed in all supported Python versions", and

the gh-77057 override shown in 4.2 waits for Python 3.13 to become the minimum. Each supported-version change in `pyproject.toml` should trigger a review of these workarounds, but nothing links the two, so they outlive the versions they were written for.

5. Remediation Cost and Debt Ratio

5.1 Method

We used the SQALE method that SonarQube implements: every finding is given a remediation effort, the efforts are summed into the remediation cost (the debt principal), and the cost is divided by the estimated cost of developing the code from scratch.

```
Remediation cost     = SUM over findings ( effort to fix the finding )
Development cost     = SLOC x 30 minutes          (SonarQube default: 30 min per line of code)
Technical Debt Ratio = Remediation cost / Development cost x 100 %
```

The effort constants are our calibration. For rule types that SonarQube also has (cognitive complexity, uncovered lines and branches, duplicated blocks) we followed the shape of its remediation functions: a fixed part plus a part that grows with the size of the violation. Findings that describe the same problem twice were costed once (for example pylint `too-many-branches` on a function already counted for cognitive complexity), and findings we accept as design decisions were given zero cost with the reason recorded (`analysis/remediation.py`).

5.2 Tool outputs and cost per item

#	Finding (tool output)	Debt type	Count	Effort rule	Minutes
R1	Over-complex functions (cognitive complexity > 15)	Code debt	26 functions	5 min + 1 min per point above 15	703
R2	Duplicated blocks	Design debt	26 clone pairs	20 min per clone pair (extract shared helper, redirect both call sites)	520
R3	Uncovered statements	Test debt	69 lines	2 min per uncovered line	138
R4	Uncovered branches	Test debt	85 branches	5 min per uncovered branch	425
R5	Version-pinned tests (Pygments == 2.7.1) that skip in a default environment	Test debt	1 test design issue	fixed estimate: 4 h to make 111 skipped tests version-tolerant	240
R6	Missing docstrings (module / class / function)	Documentation debt	62 definitions	5 min each	310
R7	Non-PEP 8 names in the public API (camelCase methods / attributes)	Code debt	85 names	15 min each (snake_case alias + deprecation shim)	1,275
R8	Non-PEP 8 local / argument / constant names	Code debt	67 names	2 min each	134
R9	Other pylint warnings / refactor / convention findings	Code debt	281 findings	1-20 min each, by rule (listed in `analysis/remediation.py`)	988
R10	Lines over the project limit of 119 columns, in 17 modules that a `flake8: noqa` comment exempts	Code debt	62 lines	1 min each	62
R11	Security findings (assert in production code; xml.etree imports to review)	Security debt	25 findings	5 asserts x 10 min + 20 import reviews x 2 min	90
R12	Oversized module `inlinepatterns.py` (1,052 lines, 44 functions, MI 25.9): split by concern	Design debt	1 module	fixed estimate: 8 h incl. re-export shim and test run	480
R13	Deprecated or dead code still shipped (10 items)	Design debt	10 items	15 min each (remove + changelog entry)	150
R14	Stale Python-version workarounds (gh-77057 override duplicated in 2 files; HTML parser TODO)	Infrastructure / dependency debt	3 workarounds	30 min each once the minimum Python allows	90
	Total remediation cost			**93.4 h = 11.7 person-days (8 h)**	**5,605**

Debt type	Minutes	Hours	Share
Code debt	3,162	52.7	56 %
Design debt	1,150	19.2	21 %
Test debt	803	13.4	14 %
Documentation debt	310	5.2	6 %
Security debt	90	1.5	2 %
Infrastructure / dependency debt	90	1.5	2 %

Worked example for R1: `HTMLExtractorExtra.handle_endtag` has cognitive complexity 94, so its effort is 5 + (94 - 15) = 84 minutes. Summed over the 26 functions above the limit this gives 703 minutes.

5.3 Technical Debt Ratio

```
Remediation cost = 5,605 min  (93.4 hours)
Development cost = 4,451 SLOC x 30 min = 133,530 min  (2,225.5 hours)
TDR              = 5,605 / 133,530 x 100 = 4.20 %
```

SQALE maintainability rating	A	B	C	D	E
Technical Debt Ratio	up to 5 %	6 - 10 %	11 - 20 %	21 - 50 %	above 50 %
Python-Markdown	4.2 %				

Justification. A ratio of 4.2 % places the project in rating A, which is consistent with the other measurements: 97 % coverage, 2.4 % duplication, no import cycles and no proven bug. The value is nevertheless close to the 5 % boundary, and three points qualify it:

- **Choice of size base.** We used SLOC (4,451), as SonarQube does. Using all 8,609 physical lines would halve the ratio to 2.2 %; the stricter base is the more honest one.
- **The debt is concentrated.** Five of the 33 modules hold 61 % of the cognitive complexity that lies in over-threshold functions, so a package-wide ratio understates how hard those files are to change.
- **The ratio measures principal only.** It does not include interest, the extra effort paid on every change. Section 6 therefore ranks the items by how often the indebted code is touched, not by cost alone.
- **Not all of it should be repaid.** R7 (1,275 minutes, 23 % of the total) is the cost of renaming camelCase API methods. That would break third-party extensions, so we report it but recommend against paying it.

6. Prioritization Strategy

Two techniques were applied. Hotspot analysis ranks where in the code debt costs the most; the weighted scoring model ranks which debt items to repay first. They use different inputs, so agreement between them is evidence rather than repetition.

6.1 Technique 1: hotspot analysis

A hotspot is code that is both complex and frequently changed, because that is where complexity is actually paid for. For every module we took the number of commits that touched it in the last three years (`git log --since="3 years ago"`, 71 commits in total) and multiplied it by the module's cognitive complexity. Commits whose message mentions a fix, bug, regression or error were counted separately as a defect indicator. `__meta__.py` (13 commits) was excluded because its changes are version-number bumps.

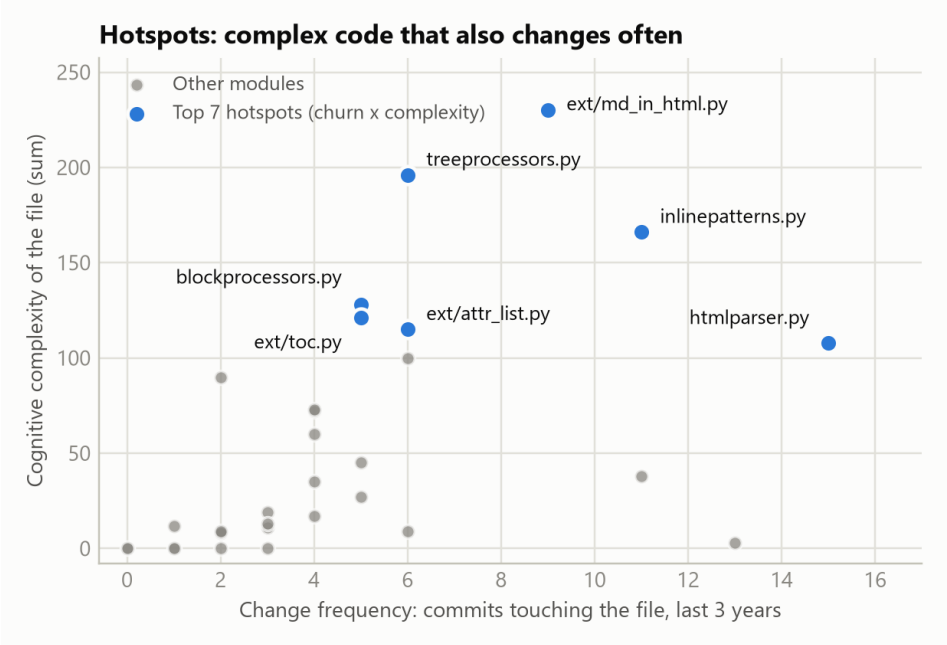


Figure 3. Change frequency against cognitive complexity per module

Rank	Module	Commits	Fixes	Cognitive	Score	Worst function (cognitive)
1	`extensions/md_in_html.py`	9	2	230	2,070	`HTMLExtractorExtra.handle_endtag (94)`
2	`inlinepatterns.py`	11	3	166	1,826	`LinkInlineProcessor.getLink (58)`
3	`htmlparser.py`	15	11	108	1,620	`HTMLExtractor.parse_starttag (19)`
4	`treeprocessors.py`	6	4	196	1,176	`InlineProcessor.__processPlaceholders (73)`
5	`extensions/attr_list.py`	6	1	115	690	`AttrListTreeprocessor.run (102)`
6	`blockprocessors.py`	5	2	128	640	`ParagraphProcessor.run (18)`
7	`extensions/toc.py`	5	1	121	605	`TocTreeprocessor.run (25)`
8	`extensions/footnotes.py`	6	1	100	600	`FootnoteExtension.findFootnotesPlaceholder (19)`

Four modules stand apart: `extensions/md_in_html.py`, `inlinepatterns.py`, `htmlparser.py` and `treeprocessors.py`. Together they hold 40 % of the package's cognitive complexity and were touched by 32 of the 71 commits. `htmlparser.py` is the clearest case of debt interest: 11 of its 15 commits were fixes. By contrast `serializers._serialize_html` is the third most complex function in the package but its file changed only twice in three years, so refactoring it would cost effort and return little.

6.2 Technique 2: weighted scoring model

Each debt item was scored from 1 (low) to 5 (high) on four criteria. The weights reflect that this is a library whose output other tools publish: user impact and defect risk together carry 55 %.

Criterion	Weight	Meaning of a high score
Impact	0.30	A failure here produces wrong or unsafe HTML for end users
Risk	0.25	The weakness is likely to cause or hide defects (fix history, missing tests)
Change frequency	0.25	The affected code is touched often, so the debt charges interest often
Ease of repayment	0.20	Low remediation cost and low risk of breaking users (5 = cheapest)

Rank	Item	Debt	Impact	Risk	Change	Ease	Score	Evidence
1	TD-01	Over-complex functions	5	5	5	2	4.40	Worst functions sit in the four hotspots; 703 min
2	TD-03	Duplicated logic	3	4	4	4	3.70	Parser fix duplicated in two hotspot files; 520 min
3	TD-04	Pinned tests and branch gaps	4	4	3	3	3.55	111 skipped tests hide regressions; 803 min
4	TD-02	Oversized `inlinepatterns.py`	3	3	5	2	3.30	11 commits in 3 years, MI 25.9; 480 min
5	TD-06	Stale version workarounds	2	3	5	3	3.20	Located in `htmlparser.py`; partly blocked until 3.13
6	TD-05	Missing documentation	2	2	4	5	3.10	17 undocumented methods in the top hotspot; 310 min
7	TD-10	Assertions as run-time checks	3	2	2	5	2.90	5 statements, 50 minutes
8	TD-07	Deprecated and dead code	2	2	2	5	2.60	150 minutes; needs a release note
9	TD-09	Lint findings, suppressed checks	1	2	3	4	2.35	Mechanical, mostly automatic fixes
10	TD-08	Naming inconsistency	2	1	2	1	1.55	Renaming breaks third-party extensions

6.3 Prioritised order and justification

The two techniques agree on the top of the list: the highest-scoring item (TD-01) is located in the modules the hotspot analysis ranks first. Combining them gives the following order.

Order	Action	Items	Effort	Why now
1	Refactor the ten over-threshold functions in the four hotspot modules, starting with `handle_endtag`, `__processPlaceholders` and `getLink`	TD-01 (part)	about 6 h	Highest score in both techniques; these functions are edited in most parser fixes
2	Move the duplicated parser workaround and the escaping helpers into one place each	TD-03 (part)	about 2 h	Cheap, and it removes the chance of a half-applied fix in the most fix-prone files
3	Make the Pygments tests version-tolerant; add tests for `smarty` and `footnotes` branches	TD-04	about 13 h	A safety net is needed before the larger refactoring in step 5
4	Quick wins: replace 5 asserts, delete dead code, document `htmlparser.py`, fix misleading comments	TD-10, TD-07, TD-05	about 9 h	High ease scores; each is under an hour per file
5	Split `inlinepatterns.py` by concern behind a re-export shim; refactor remaining complex functions	TD-02, TD-01 (rest)	about 14 h	Valuable but risky; depends on step 3
6	Remove version workarounds when the minimum Python version reaches 3.13	TD-06	1.5 h	Blocked by the support policy; add a checklist item to the release process
7	Fix lint findings opportunistically when a file is edited anyway	TD-09	about 17 h	No user impact; not worth dedicated effort
-	Do not rename the camelCase public API	TD-08	(21 h avoided)	Cost and breakage for every third-party extension outweigh a cosmetic benefit

Steps 1 to 4 need about 30 hours, one third of the total remediation cost, and address the items that both techniques place highest.

7. Reflection

Lessons learned

- **Averages hide debt.** A mean cyclomatic complexity of 3.3, 97 % coverage and an A rating describe a healthy project, and all three are true. The debt only became visible when we looked at distributions: 6 % of the functions carry the complexity problem, and four files carry most of the change effort.
- **One metric is not enough.** `getLink` and `AttrListTreeprocessor.run` have nearly the same cyclomatic complexity but very different cognitive complexity. Coverage was 94 % or 97 % depending on an environment variable. Each number needed a second measurement to interpret it.

- **Tools disagree, and the reasons are instructive.** The cognitive-complexity tool counts list comprehensions and the published specification does not; pylint counted 196 long lines against a limit the project does not use. Doing the calculations by hand (Appendix A) is what exposed these rule differences.
- **History is a metric.** The git log turned a list of complex functions into a ranking. Without change frequency we would have put `_serialize_html` near the top of the list, and it has barely changed in three years.

Trade-offs considered

- **Compatibility against cleanliness.** Much of what the tools flag (camelCase names, legacy extensions, deprecated shims) exists because third-party extensions depend on it. For a library, some debt is a deliberate and reasonable choice; we costed it but recommended against repaying part of it.
- **Refactoring risk against benefit.** The functions most in need of refactoring are in the HTML parser, where 11 of 15 recent commits were fixes. Refactoring there is the most valuable and the most dangerous work, which is why test debt is ranked ahead of the larger structural change.
- **Precision of the cost model.** Our remediation constants are estimates. A different calibration would move the 4.2 % ratio by a point either way, so we rely on the ranking, which depends on relative values, more than on the absolute figure.
- **Tool choice.** Not using SonarQube cost us an integrated dashboard, but assembling the measurements ourselves forced us to understand what each one counts.

Implications for maintenance and evolution

- Debt should be managed where change happens. A rule such as "do not increase cognitive complexity in a hotspot file" enforced in CI would stop the debt growing at almost no cost.
- Workarounds need an expiry mechanism. Linking each one to the minimum supported Python version in the release checklist would prevent them from outliving their purpose.
- Suppressions are debt too. File-wide `noqa` directives and 88 `pragma: no cover` markers make the quality gates pass while reducing what they check; they should be reviewed like code.
- For a twenty-year-old project maintained by a few volunteers, the realistic goal is not zero debt but keeping the interest payments low in the files that still evolve.

Appendix A. Individual Component: Manual Calculations

Each member selected two non-trivial files and calculated cyclomatic complexity, cognitive complexity and comment density by hand. The scanned handwritten sheets follow each member's summary. The rules used were the same for all six files:

- **Cyclomatic complexity** (McCabe): CC = decision points + 1 per function, counting `if`, `elif`, `for`, `while`, `except`, each `and` / `or`, each comprehension `for` and `if`, and conditional expressions. A nested function is a separate unit.
- **Cognitive complexity** (SonarSource specification): +1 for each `if`, `elif`, `else`, loop, `except`, conditional expression and each sequence of like boolean operators; an extra +1 per nesting level for `if`, loops, `except` and conditional expressions. A nested function is charged to its parent, one level deeper.
- **Comment density** (SonarQube definition): comment lines / (comment lines + code lines) x 100, where comment lines are # comment lines plus docstring lines, and blank lines are excluded.

A.1 Member 1: Taha Yasir (i222429)

File	Lines	Blank	# comment	Docstring	Code	Comment density
`extensions/def_list.py`	128	29	18	18	63	36 / (36 + 63) = 36.4 %
`extensions/meta.py`	93	22	11	13	47	24 / (24 + 47) = 33.8 %

File	Function	Cyclomatic	Cognitive
`extensions/def_list.py`	`DefListProcessor.run`	17	17
	`4 other functions with no decision point`	4 x 1	0
	`File total (hand) | tool cross-check`	21 | 21	17 | 19
`extensions/meta.py`	`MetaPreprocessor.run`	10	17
	`3 other functions with no decision point`	3 x 1	0
	`File total (hand) | tool cross-check`	13 | 13	17 | 17

Interpretation: extensions/def_list.py

Observed risks. Almost all of the file's complexity is in one method. `DefListProcessor.run` has a cyclomatic complexity of 17 (rank C) while the other four functions have 1. The count is driven by compound conditions rather than by structure: six `and` operators, including `if not terms and len(dl) and dl[-1].tag == 'dd' and len(dl[-1])` on line 74. The method also tracks a `state` variable that is assigned in three places, and line 61 reads `sibling.tag` safely only because an earlier branch has already returned when `sibling` is `None`. It returns `False` on one path and nothing on the others.

Debt and maintenance. This is code debt with a documentation side: the comment density of 36.4 % looks healthy, but 15 of the 36 comment lines are the licence header and a registration table, and the only complex method has no docstring. Seventeen independent paths need at least seventeen test cases; statement coverage is 96.6 %. The file changed three times in three years, so the interest is low. Extracting the term-splitting and the list-lookup into two helpers would bring `run` under 10 at little risk. The tool reports cognitive complexity 19 where the hand calculation gives 17, because the tool also counts the list comprehension on lines 43-44.

Interpretation: extensions/meta.py

Observed risks. `MetaPreprocessor.run` is a hand-written line parser: a `while` loop with two `break` statements, a `try / except KeyError` used for normal control flow, and three levels of nesting. Its cognitive complexity (17) is much higher than its cyclomatic complexity (10), which signals that nesting, not the number of branches, makes it hard to read. It consumes its input with `lines.pop(0)`, which is linear per call. Coverage is 100 % for statements but 90 % for branches: the case where the document ends inside the metadata block is not tested.

Debt and maintenance. The more important finding is design debt. `MetaExtension.reset` creates the attribute `Markdown.Meta` on an object it does not own, and `wikilinks.py` reads that attribute with `hasattr`. Two extensions are therefore coupled through an attribute that neither the `Markdown` class nor the import graph declares. A maintainer changing the metadata format has no way to discover the second reader. Using `dict.setdefault` instead of the exception, and declaring `Meta` on the core class, would remove both issues.

[Insert Member 1's scanned handwritten calculation sheets after this page]

A.2 Member 2: Khizar (i222595)

File	Lines	Blank	# comment	Docstring	Code	Comment density
extensions/admonition.py	192	48	27	19	98	46 / (46 + 98) = 31.9 %
extensions/wikilinks.py	105	25	10	15	55	25 / (25 + 55) = 31.2 %

File	Function	Cyclomatic	Cognitive
extensions/admonition.py	AdmonitionProcessor.parse_content	13	19
	AdmonitionProcessor.test	2	2
	AdmonitionProcessor.run	8	12
	AdmonitionProcessor.get_class_and_title	3	2
	3 other functions with no decision point	3 x 1	0
	File total (hand) | tool cross-check	29 | 29	35 | 35
extensions/wikilinks.py	WikiLinksInlineProcessor.handleMatch	3	4
	WikiLinksInlineProcessor._getMeta	5	7
	5 other functions with no decision point	5 x 1	0
	File total (hand) | tool cross-check	13 | 13	11 | 11

Interpretation: extensions/admonition.py

Observed risks. `parse_content` (cyclomatic 13, cognitive 19) contains a `while` loop holding an `if` with a four-part `and` and, inside it, a conditional expression at nesting level 3 that alone costs four cognitive points. Some of its conditions are redundant (`last_child is not None` is re-tested inside a loop that already guarantees it). The class also carries state between two methods: `test()` stores `current_sibling` and `content_indent`, and `run()` relies on them, so the methods are only correct when called in that order. In `run`, `if m:` is tested twice in a row, and the second `else` uses a variable bound in the first.

Debt and maintenance. This is code debt combined with design debt (a hidden calling protocol). The class and three of its methods have no docstrings, and the comment on line 159 is garbled, so the density of 31.9 % overstates how well the logic is explained. On the positive side, statement and branch coverage are both 100 %, which makes refactoring safe: merging the two `if m` blocks and extracting the list-sibling search would be a low-risk first step.

Interpretation: extensions/wikilinks.py

Observed risks. Complexity is low: every function is rank A and the highest cyclomatic value is 5. The risks are structural. `_getMeta` contains three near-identical `if 'wiki_...' in self.md.Meta` blocks, a small copy-paste clone that a loop over the three keys would replace. `handleMatch` returns an `Element` on one path and an empty string on the other. The `md` reference is attached after construction (`wikilinkPattern.md = md`) instead of being passed to the constructor.

Debt and maintenance. The file depends on the `meta` extension through the undeclared `Markdown.Meta` attribute, the mirror image of the coupling found in `meta.py`. That is mild design debt. Repaying everything here would take about half an hour, and the file changed three times in three years, so it is rightly low on the priority list. It is a useful baseline: this is what a low-debt extension in this project looks like.

[Insert Member 2's scanned handwritten calculation sheets after this page]

A.3 Member 3: [Member 3 name] (i228780)

File	Lines	Blank	# comment	Docstring	Code	Comment density
postprocessors.py	131	35	14	24	58	38 / (38 + 58) = 39.6 %
extensions/legacy_attrs.py	78	21	14	16	27	30 / (30 + 27) = 52.6 %

File	Function	Cyclomatic	Cognitive
postprocessors.py	RawHtmlPostprocessor.run	2	10
	RawHtmlPostprocessor.run > substitute_match	5	(in parent)
	RawHtmlPostprocessor.isblocklevel	3	3
	6 other functions with no decision point	6 x 1	0
	File total (hand) | tool cross-check	16 | 16	13 | 13
extensions/legacy_attrs.py	LegacyAttrs.run	7	9
	4 other functions with no decision point	4 x 1	0
	File total (hand) | tool cross-check	11 | 11	9 | 9

Interpretation: postprocessors.py

Observed risks. `RawHtmlPostprocessor.run` hides its logic in a nested function, `substitute_match`, which calls itself indirectly through `pattern.sub(substitute_match, html)`. The recursion depth depends on how raw-HTML placeholders are nested in the input document. The closure also uses `pattern`, which is defined after it in the enclosing function. Per-function tools understate this method: radon reports 2 for `run` and 5 for the closure separately, while cognitive complexity charges the closure to its parent one level deeper and gives 10.

Debt and maintenance. The file carries obsolescence debt: `UnescapePostprocessor` is marked `@deprecated` but still shipped, and its two methods are the file's only uncovered lines. It also has documentation debt of the misleading kind: three docstrings contain errors (lines 22, 49 and 96), and the base method `Postprocessor.run` is a bare `pass`, so a subclass that forgets to override it silently returns `None`. The second-highest comment density of the six files (39.6 %) therefore does not mean good documentation.

Interpretation: extensions/legacy_attrs.py

Observed risks. The file has only 27 code lines. `LegacyAttrs.run` reaches a cyclomatic complexity of 7 from three `if` statements, two of them with an `and`, all inside one loop. `handleAttributes` changes the element as a side effect inside a regular-expression callback, and the callback returns nothing: it relies on `re.sub` treating `None` as an empty replacement.

Debt and maintenance. The whole module is a deliberate debt: it exists only so that documents written before version 3.0 keep rendering. Its comment density of 52.6 % is the highest we measured, but 13 of the 30 comment lines are the licence header, and neither class has a docstring. It uses absolute imports (`from markdown.treeprocessors import ...`) where every other extension uses relative ones. The cost of keeping it is small but permanent: it must be tested on every new Python version until a major release removes it.

[Insert Member 3's scanned handwritten calculation sheets after this page]

Appendix B. Reproducing the Measurements

All measurements were taken on commit `0bf535bc406c` with Python 3.13 on Windows 11. Raw tool outputs and the scripts that aggregate them are in the `analysis/` folder submitted with the project copy.

```
git clone https://github.com/Python-Markdown/markdown.git && cd markdown
git checkout 0bf535bc406c95dad66050f07c8402af03fe07d5


radon raw markdown -j        # size, comment lines
radon cc  markdown -s -a -j  # cyclomatic complexity
radon mi  markdown -s -j     # maintainability index
complexipy markdown --output-format json           # cognitive complexity
npx jscpd markdown --min-lines 5 --min-tokens 50    # duplication
pylint markdown --output-format=json2 --exit-zero   # bugs / code smells
bandit -r markdown -f json                          # vulnerabilities
flake8 markdown --max-line-length=119 ; vulture markdown --min-confidence 80

# coverage: two runs, combined (statement + branch); .coveragerc set aside so test_tools.py is measured too
coverage run -p --branch --source=markdown -m unittest discover tests      # without Pygments
pip install pygments==2.7.1 && set PYGMENTS_VERSION=2.7.1
coverage run -p --branch --source=markdown -m unittest discover tests      # with pinned Pygments
coverage combine && coverage report


git log --since="3 years ago" --name-only --format=@%H^|%s -- markdown     # change frequency
```

Script	Purpose
`analysis/summarize.py`	Merges all tool outputs into `summary.json` and `per_file_metrics.csv`
`analysis/deps.py`	Builds the import graph, fan-in / fan-out, instability and cycle check
`analysis/remediation.py`	Applies the effort rules of Section 5 and computes the debt ratio
`analysis/manual_calc.py`	Step-by-step check of the hand calculations in Appendix A
`analysis/charts.py`	Draws Figures 1 to 3

References

- T. J. McCabe, "A Complexity Measure", IEEE Transactions on Software Engineering, SE-2(4), 1976.
- G. A. Campbell, "Cognitive Complexity: A new way of measuring understandability", SonarSource, 2018.
- J.-L. Letouzey, "The SQALE Method for Managing Technical Debt", 2016.
- A. Tornhill, "Your Code as a Crime Scene", Pragmatic Bookshelf, 2015 (hotspot analysis).
- Python-Markdown documentation, https://python-markdown.github.io/